

ALGORITHM

1. Main Algorithm

1. Start the program.
2. Declare an array `arr` of size 100 and variables `n` (number of elements), `choice`, `key`, `pos`, `val`, `k`.
3. Input the initial number of elements `n` and the array elements.
4. Display the menu:
 - 1. Bubble Sort and Binary Search
 - 2. Insertion and Deletion
 - 3. Rotate Array Right by `k` positions
 - 4. Exit
5. Read user's choice:
 - **If choice = 1:** Call Bubble Sort function to sort the array. Print sorted array. Then input `key` to search and call Binary Search function.
 - **If choice = 2:** Ask user whether to Insert (1) or Delete (2).
 - If Insert: Read position and value. Check if position is valid (1 to `n+1`). Call Insertion function.
 - If Delete: Read position. Check if position is valid (1 to `n`). Call Deletion function.
 - **If choice = 3:** Input number of positions `k` to rotate. Call Right Rotate function. Print modified array.
 - **If choice = 4:** Exit the program.
 - **Default:** Print invalid choice.
6. Repeat step 4 and 5 until user enters choice 4.
7. Stop.

2. Bubble Sort Algorithm

1. Loop `i` from 0 to `n - 2`.
2. Loop `j` from 0 to `n - i - 2`.
3. If `arr[j] > arr[j + 1]`, swap `arr[j]` and `arr[j + 1]`.
4. Repeat until array is sorted.

3. Binary Search Algorithm

1. Set `low = 0` and `high = n - 1`.

2. While low <= high:
 - Set mid = (low + high) / 2.
 - If arr[mid] == key, return mid (found).
 - If arr[mid] < key, set low = mid + 1.
 - Else set high = mid - 1.
3. If not found, return -1.

4. Insertion Algorithm

1. Shift elements from index n - 1 down to pos - 1 one position right:
arr[i + 1] = arr[i].
2. Insert val at index pos - 1: arr[pos - 1] = val.
3. Increase size n = n + 1.

5. Deletion Algorithm

1. Shift elements from index pos up to n - 1 one position left: arr[i - 1] = arr[i].
2. Decrease size n = n - 1.

6. Right Rotation Algorithm

1. Set k = k % n.
2. Repeat k times:
 - Store last element in temporary variable temp = arr[n - 1].
 - Shift all elements right by 1 position: arr[i] = arr[i - 1] for i = n - 1 down to 1.
 - Set arr[0] = temp.

C PROGRAM CODE

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```

}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while(low <= high) {
        mid = (low + high) / 2;
        if(arr[mid] == key) {
            return mid;
        }
        else if(arr[mid] < key) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
    return -1;
}

void insertElement(int arr[], int *n, int pos, int val) {
    int i;
    for(i = *n - 1; i >= pos - 1; i--) {
        arr[i + 1] = arr[i];
    }
    arr[pos - 1] = val;
    (*n)++;
}

void deleteElement(int arr[], int *n, int pos) {
    int i;
    for(i = pos; i < *n; i++) {
        arr[i - 1] = arr[i];
    }
    (*n)--;
}

void rotateRight(int arr[], int n, int k) {
    int i, j, temp;
    k = k % n;
    for(i = 0; i < k; i++) {
        temp = arr[n - 1];
        for(j = n - 1; j > 0; j--) {
            arr[j] = arr[j - 1];
        }
        arr[0] = temp;
    }
}

void printArray(int arr[], int n) {
    int i;
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("

```

```

");
}

int main() {
    int arr[100], n, choice, subChoice, key, pos, val, k, res;

    printf("Enter number of elements in array: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for(int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    while(1) {
        printf("
--- MENU ---
");
        printf("1. Sort (Bubble Sort) and Binary Search
");
        printf("2. Insertion and Deletion
");
        printf("3. Rotate Array Right by k positions
");
        printf("4. Exit
");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if(choice == 1) {
            bubbleSort(arr, n);
            printf("Array after Bubble Sort: ");
            printArray(arr, n);

            printf("Enter element to search using Binary Search: ");
            scanf("%d", &key);
            res = binarySearch(arr, n, key);
            if(res != -1) {
                printf("Element %d found at index %d (position %d).
", key, res, res + 1);
            } else {
                printf("Element %d not found in array.
", key);
            }
        }
        else if(choice == 2) {
            printf("1. Insert Element
");
            printf("2. Delete Element
");
            printf("Enter sub-choice: ");
            scanf("%d", &subChoice);

            if(subChoice == 1) {

```

```

        printf("Enter position (1 to %d): ", n + 1);
        scanf("%d", &pos);
        if(pos < 1 || pos > n + 1) {
            printf("Invalid position!
");
        } else {
            printf("Enter value to insert: ");
            scanf("%d", &val);
            insertElement(arr, &n, pos, val);
            printf("Array after insertion: ");
            printArray(arr, n);
        }
    }
    else if(subChoice == 2) {
        printf("Enter position to delete (1 to %d): ", n);
        scanf("%d", &pos);
        if(pos < 1 || pos > n) {
            printf("Invalid position!
");
        } else {
            deleteElement(arr, &n, pos);
            printf("Array after deletion: ");
            printArray(arr, n);
        }
    }
    else {
        printf("Invalid sub-choice!
");
    }
}
else if(choice == 3) {
    printf("Enter number of positions to rotate right (k): ");
    scanf("%d", &k);
    rotateRight(arr, n, k);
    printf("Array after rotating right by %d positions: ", k);
    printArray(arr, n);
}
else if(choice == 4) {
    printf("Exiting program...
");
    break;
}
else {
    printf("Invalid choice! Try again.
");
}
}

return 0;
}

```

TEST CASES

Test Case 1: Option 1 - Bubble Sort & Binary Search

Input :

Initial Array: [5, 2, 8, 1, 3] (size n = 5)

Choice: 1

Search Key: 8

Expected Output:

Sorted Array: 1 2 3 5 8

Search Result: Element 8 found at index 4 (position 5).

Test Case 2: Option 2 - Insertion

Input :

Current Array: [1, 2, 3, 5, 8] (size n = 5)

Choice: 2 -> Sub-choice: 1

Position: 4, Value: 4

Expected Output:

Updated Array: 1 2 3 4 5 8 (size n = 6)

Test Case 3: Option 2 - Deletion

Input :

Current Array: [1, 2, 3, 4, 5, 8] (size n = 6)

Choice: 2 -> Sub-choice: 2

Position: 2

Expected Output:

Updated Array: 1 3 4 5 8 (size n = 5)

Test Case 4: Option 3 - Right Rotation

Input :

Current Array: [1, 3, 4, 5, 8] (size n = 5)

Choice: 3

k positions: 2

Expected Output:

Updated Array: 5 8 1 3 4